Special Numbers

Company: Accenture
Round: Online Assessment (OA) — Cognitive and Technical Assessment
Year: 2023
Difficulty: Easy
Topic: Arrays, Number Theory, Divisibility
Source: LeetCode Discuss — Accenture OA (Cognitive and Technical Assessment 2023)

Problem Description

You are given an array Arr of N positive integers (none of which is 1) and an integer K. You need to find the K smallest positive integers that are not divisible by any of the N integers in Arr, and return the sum of those K integers.

Input Format

- The first line contains two space-separated integers, N and K.
- The second line contains N space-separated integers denoting the array Arr.

Output Format

- Print a single integer — the sum of the K smallest positive integers not divisible by any element of Arr.

Constraints

- $1 \leq N \leq 100$
- $1 \leq K \leq 1000$
- $2 \leq Arr[i] \leq 1000$ (Arr never contains 1)

Sample Input / Output

Example 1
Input:
5 4
2 3 4 5 6
Output:
32

Example 2
Input:
4 4
3 6 9 12
Output:
12

Explanation

In Example 1, checking integers one by one starting from 1: 1 is not divisible by any of 2, 3, 4, 5, or 6, so it qualifies. 2 through 6 are each divisible by at least one array element, so they're skipped. 7 qualifies (not divisible by any). 8 through 10 are skipped. 11 qualifies. 12 is skipped. 13 qualifies. That gives the four smallest valid numbers: 1, 7, 11, 13 → sum = 32.

In Example 2, 1 qualifies (not divisible by 3, 6, 9, or 12). 2 qualifies. 3 is divisible by 3, so it's skipped. 4 qualifies. 5 qualifies. That gives 1, 2, 4, 5 → sum = 12.

Approach to Solve This Problem:

Since Arr never contains 1, the number 1 is always guaranteed to be a valid candidate (nothing greater than 1 can divide it), which also guarantees the search always finds at least one valid number and never gets stuck. Beyond that, this is a straightforward candidate-generation problem: test each positive integer starting from 1 in increasing order, check whether it's divisible by any element of Arr, and collect it if it isn't — stopping once K valid numbers have been found.

1. **Initialize:** Start a candidate counter at 1, a count of valid numbers found at 0, and a running sum at 0.
2. **Test each candidate:** For the current candidate, check whether it is divisible by any element of Arr (candidate % Arr[i] == 0 for some i).
3. **Collect valid candidates:** If the candidate is not divisible by any element of Arr, add it to the running sum and increase the count of valid numbers found.
4. **Repeat until done:** Move to the next candidate and repeat steps 2–3 until K valid numbers have been collected.
5. **Return the sum:** Once K valid numbers have been found, return the accumulated sum.

Java Solution:

```java
import java.util.Scanner;

class Solution {
    public static int desiredArray(int[] arr, int n, int k) {
        long sum = 0;
        int count = 0;
        int candidate = 1;

        while (count < k) {
            boolean isDivisible = false;
            for (int i = 0; i < n; i++) {
                if (candidate % arr[i] == 0) {
                    isDivisible = true;
                    break;
                }
            }
            if (!isDivisible) {
                sum += candidate;
                count++;
            }
            candidate++;
        }

        return (int) sum;
    }
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int k = sc.nextInt();

        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        int result = desiredArray(arr, n, k);
        System.out.println(result);
    }
}
```

Solution:

1. **Read the input:** main() reads N and K via Scanner, then reads N integers into arr, matching the Input Format.
2. **Initialize counters:** desiredArray() sets sum to 0, count to 0, and candidate to 1, ready to start testing integers from 1 upward.
3. **Check divisibility:** For each candidate, the inner for loop checks candidate % arr[i] == 0 against every element of arr, setting isDivisible to true and breaking early as soon as a match is found.
4. **Collect valid candidates:** If isDivisible remains false after checking all of arr, candidate is added to sum and count is incremented.
5. **Repeat and print:** candidate is incremented each iteration, and the while loop continues until count reaches k. Once k valid numbers have been found, desiredArray() returns sum, and main() prints it directly — matching the Output Format.

Complexity Analysis:

Time Complexity: O(X × N)

where X is the value of the K-th valid number found. In the worst case, every candidate from 1 up to X is tested against all N elements of Arr, giving O(X × N) time. Since X grows with both K and the density of "forbidden" multiples in Arr, this is efficient for typical OA input sizes but is not asymptotically optimal for very large K or highly restrictive arrays.

Space Complexity: O(1)

Excluding the input array itself, the algorithm only uses a fixed number of scalar variables (sum, count, candidate), so no additional space is required.